Transformers

Lesson 3: Architecture Variations — BERT, GPT, T5, and Mixture-of-Experts

Where Lessons 1–2 left us

Two lessons built **one** machine:

- **L1** — attention: query/key/value, scaled dot-product, multi-head, the worked English→Spanish translation, end to end.
- **L2** — anatomy: input, encoder block, decoder block (the causal mask's math), output head, the four parameter-count formulas, and how position gets injected.

> Today: that **one design** — attention, FFN, and the residual+LN wrapper you saw around every sublayer — gets **reassembled into three different machines**, each tuned for a different job.

Agenda

- **BERT** — encoder-only, for understanding
- **GPT** — decoder-only, for generation
- **T5** — encoder-decoder, text-to-text
- **The family tree**, tabulated
- **Mixture-of-experts** — cheap capacity, orthogonal to the axis above
- **Board exercise** L3-a — pick the family
- **Homework briefing**

Learning Objectives

By the end of this lesson, you'll be able to:

- **Match a task to an architecture family**
- **Name each family's pretraining objective**
- **Explain MoE routing**

Part I — The family axis

One design, three shapes

The same three ingredients — **attention**, **feed-forward network (FFN)**, and the **residual + layer-norm wrapper** around every sublayer — recombine into three architectures, each tuned for a different job.

BERT

keep the *encoder* — bidirectional, for *understanding*

GPT

keep the *decoder* — causal, for *generating*

T5

keep *both* — anything cast as text → text

A brief history

- **2017** — The **Transformer** — "Attention Is All You Need" (Vaswani et al.), one encoder-decoder design.
- **2018** — **BERT** (Devlin et al.) and **GPT** (Radford et al.) — the fork: encoder-only vs decoder-only.
- **2019** — **RoBERTa**, **DistilBERT**, **GPT-2** — refine, shrink, scale.
- **2020** — **ALBERT**, **ELECTRA**, **T5** (Raffel et al.), **BART** — parameter tricks and text-to-text.
- **2021** — **Switch Transformer** (Fedus et al.) — mixture-of-experts past a trillion parameters.
- **2022** — **Chinchilla** (Hoffmann et al.) — compute-optimal scaling.
- **2023** — **LLaMA** + **GQA** (Touvron et al.; Ainslie et al.) — open decoder-only at scale.

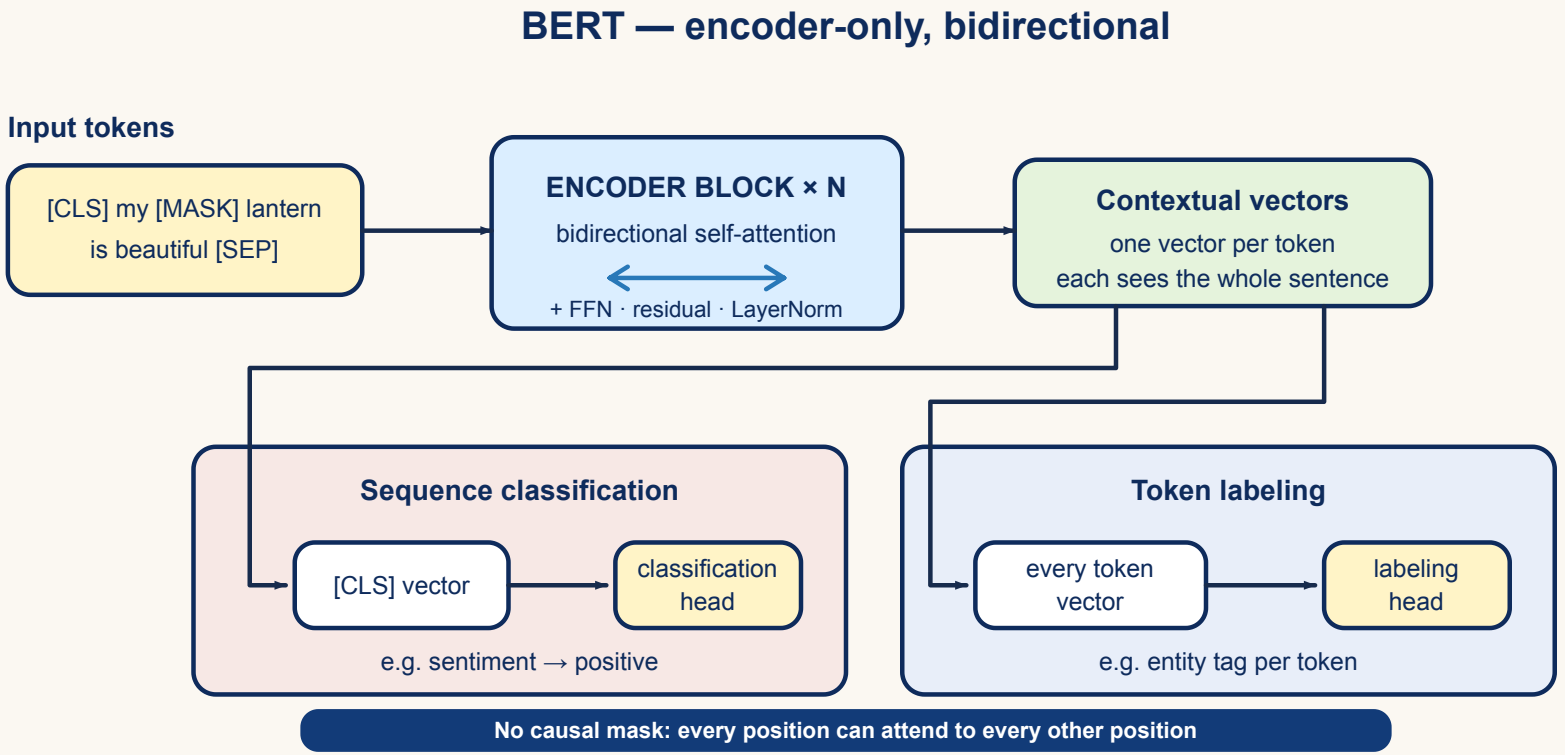
Part II — BERT: encoder-only

(a) BERT: keep the encoder, read bidirectionally

Bidirectional **E**ncoder **R**epresentations from **T**ransformers: a stack of N **encoders** with **bidirectional** self-attention — every token sees the whole sentence, **past and future** at once.

> **Why bidirectional?** For *reading* tasks — sentiment, entities, entailment — a word's meaning depends on context from **both** sides. There is no causal mask anywhere in BERT.

BERT, drawn



Bidirectional attention throughout; the two downstream heads shown below are filled in once we reach fine-tuning.

Special tokens

Token	Purpose
`[CLS]`	Prepended to every input; its final embedding is the **whole-sentence summary**, used in downstream tasks
`[SEP]`	Separates two segments defined by the task
`[MASK]`	Marks a token the model must predict at training time

BERT also keeps a handful of **`unused`** tokens: during fine-tuning they can be repurposed to learn a representation specific to a new domain.

BERT's pretraining game: Masked Language Modeling

Masked Language Modeling (MLM) — randomly mask ~15% of the tokens and predict them from **both** sides' context. But not every masked position becomes a literal `[MASK]`:

80%	10%	10%
replaced with `[MASK]`	replaced with a *random* token	left *unchanged*

Why: if every masked position were literally `[MASK]`, the model would only learn to fill that one token and could get lazy everywhere else — the 80/10/10 split forces it to build a good representation at **every** position, since it never knows in advance which positions will be graded.

Made concrete: masking one sentence

Take a 20-token sentence; **15% of 20 = 3** positions are selected for prediction:

> `my old paper lantern is glowing over the wooden table in the quiet room while the rain taps the window`

Position	Original token	What the model sees	Target
4	`lantern`	`[MASK]` *(the 80% case)*	`lantern`
10	`table`	`bicycle` *(the 10% random case)*	`table`
17	`rain`	`rain` *(the 10% unchanged case)*	`rain`

All three targets are the **original** token, whatever the input shows.

A second objective: Next-Sentence Prediction

Next-Sentence Prediction (NSP) — via the `[CLS]` vector, predict whether sentence B genuinely follows sentence A in the source text, or is a random pairing.

> Both objectives train **at once**: MLM teaches word-in-context; NSP teaches sentence-to-sentence coherence, which downstream tasks like question-answering rely on.

BERT's downstream job: fine-tune a small head

Pretrain **once** (MLM + NSP, on unlabeled text); then **add a small head** and fine-tune on a little labeled data.

Two head types

- On `[CLS]` → **sentence classification** (e.g. sentiment).
- On **each token** → **token labeling** (e.g. named-entity recognition, NER).

What moves, what doesn't

- The **pretrained weights** move a little (fine-tuning).
- The **head** is new, small, and randomly initialized.

Made concrete: how small is "a small head"?

DistilBERT-base = $N{=}6, d_{\text{model}}{=}768, h{=}12$ → **66M** parameters.

A binary sentiment head is one linear layer $768 \to 2$ on the `[CLS]` vector:

Piece	Count
Head weights, 768×2	1,536
Head biases	2
New parameters	**1,538**
Share of the 66M model	**0.002%**

Essentially **everything** the classifier knows was learned during pretraining; fine-tuning only nudges it toward one task.

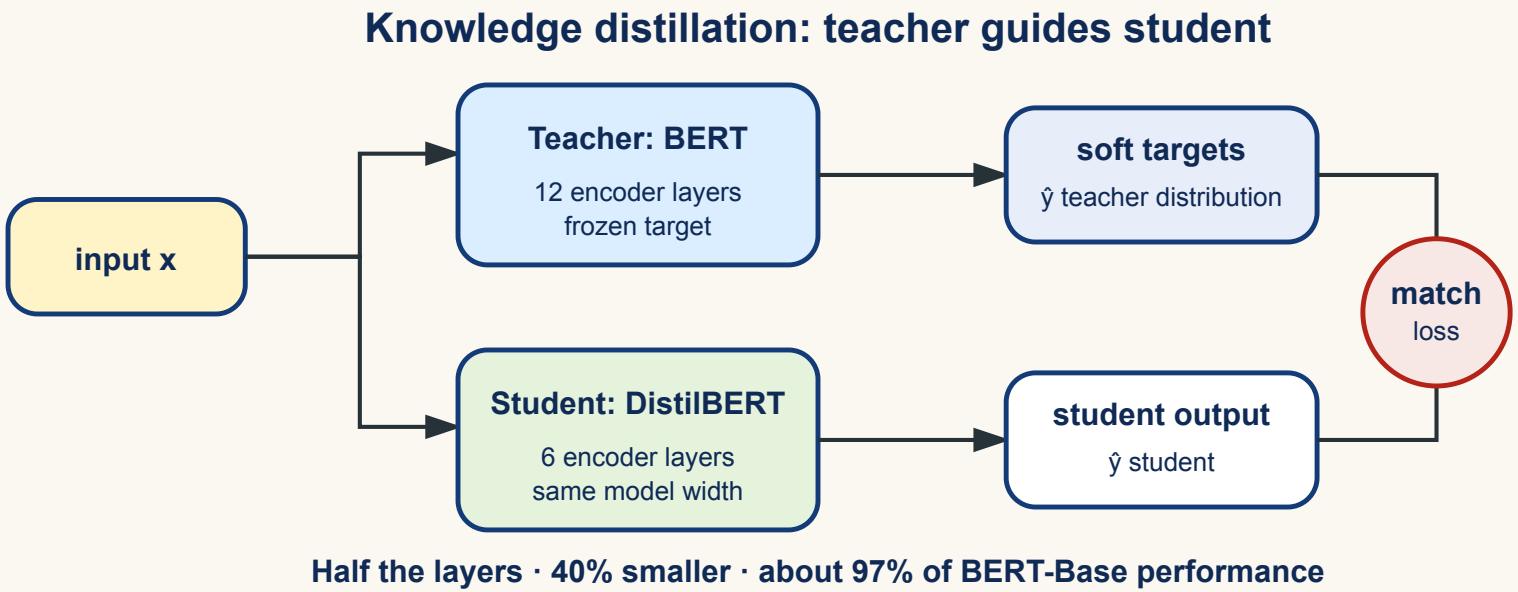
BERT variants, at a glance

Variant	One-line idea
DistilBERT	distilled: 40% smaller, ~97% of the performance
RoBERTa	more data, no NSP, dynamic masking
ALBERT	parameter sharing + factorized embeddings
ELECTRA	replaced-token detection instead of MLM — trains on all tokens

Four different bets on the same encoder-only shape. The next four slides open each one up.

DistilBERT — teach a small model to mimic a big one

Distillation: train a *student* (DistilBERT) to match a *teacher* (BERT) — not just the ground-truth labels, but the teacher's own **output distribution** (soft labels).



RoBERTa — same shape, better training recipe

Same encoder-only architecture as BERT. What changes is entirely **how** it's trained:

- **More data** — a much bigger pretraining corpus.
- **Objective simplified** — MLM only, with **dynamic masking** (a fresh random mask each epoch, instead of one fixed mask computed once); **NSP dropped** entirely.
- **Bigger batches, longer sequences** during training.
- **BPE tokenizer** instead of WordPiece.

> ⚠️ **The lesson:** none of BERT's architecture changed — RoBERTa shows how much performance was left on the table by *how* BERT was trained, not by its shape.

ALBERT — the same job, far fewer parameters

Unlike RoBERTa (same architecture, better recipe), **ALBERT changes the architecture itself**: it **shares one parameter set across all N layers** — the same W^Q, W^K, W^V, W^O and the same FFN, applied N times in sequence.

Base config, $d_{\text{model}}=768$, $d_{\text{ff}}=3072$	BERT-base	ALBERT-base
Attention per layer, $4d_{\text{model}}^2$	2.36M	2.36M
FFN per layer, $2d_{\text{model}}d_{\text{ff}}$	4.72M	4.72M
Whole $N=12$ stack	**85M**	**7.08M** — one shared set

⚠️ **Sharing weights saves memory, not compute** — all N layers still run, so FLOPs per token are unchanged. Contrast **DistilBERT**, which *removes* layers; ALBERT keeps all N and reuses their weights.

ELECTRA — every token teaches the model

BERT's MLM only produces a training signal at the **masked ~15%** of positions. **ELECTRA** replaces MLM with **Replaced Token Detection (RTD)**, which grades *every* token:

1. Take the input; replace ~15% of tokens with `[MASK]` (same as MLM's setup).
2. A small **generator** — itself a tiny language model — fills each `[MASK]` with a **plausible** token. The sequence now has **no `[MASK]` tokens left**, just a mix of original and generator-filled words.
3. A **discriminator** reads the *whole* modified sequence and predicts, **for every token**, original or replaced.
4. At inference, only the **discriminator** is kept for downstream tasks.

BERT's size ladder

Version	Parameters	d_{model}	d_{ff}	h	N
Tiny	4M	128	512	2	2
Mini	11M	256	1,024	4	4
Small	30M	512	2,048	8	4
Medium	42M	512	2,048	8	8
Base	110M	768	3,072	12	12
Large	340M	1,024	4,096	16	24

Same **four hyperparameters** as Lesson 2's anatomy card ($d_{\text{model}}, d_{\text{ff}}, h, N$) — a whole model family is just six rows of these four numbers.

BERT: strengths & limitations

- ✅ **Bidirectional context** — the right shape for reading tasks: classification, extraction, entailment.
- ✅ **Pretrain once, fine-tune cheaply** — a ~1,500-parameter head plus a small labeled set gets you a task model.
- ⚠️ **Cannot generate** left-to-right text: no causal decoder. BERT *reads*; it does not *write*.

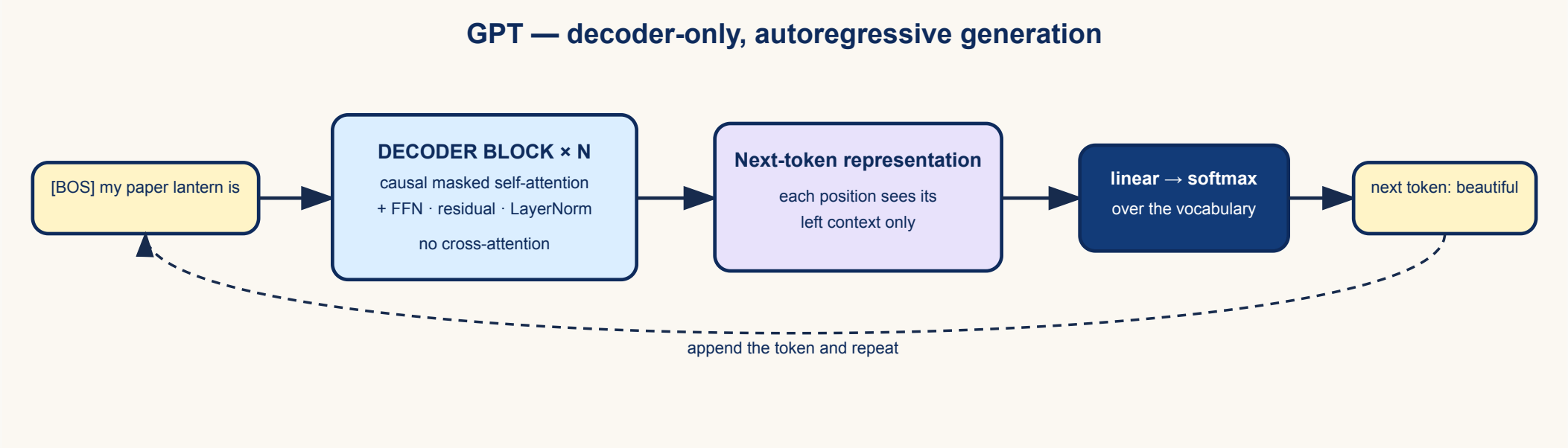
> ⚠️ **Misconception:** *"BERT is old / worse than GPT."* Different **jobs**. BERT is still excellent — and markedly **cheaper** — for classification and extraction. The homework's H3 puts this to the test directly.

Part III — GPT: decoder-only

(b) GPT: keep the decoder, read causally

Generative **P**re-trained **T**ransformer: a stack of N **decoders**, each with **only** masked (causal) self-attention — **no cross-attention**, since there's no separate encoder to attend to. Every token attends to its **left** context only.

GPT, drawn



The dashed loop is the whole story: **generate one token, feed it back in, repeat** — Lesson 2's decoder loop, now with no encoder to fall back on.

GPT's pretraining game: causal / next-token LM

Predict token $t+1$ from tokens $1..t$ — cross-entropy, at scale.

> Because the causal mask lets every position predict-the-next **in parallel** (Lesson 2's mask math), **one sequence of length n gives n training signals** — one per position — in a single forward pass.

GPT's size ladder

Version	Parameters	d_{model}	d_{ff}	h	N	Context
GPT-1	117M	768	3,072	12	12	512
GPT-2	1.5B	1,600	3,072	25	48	1,024
GPT-3	175B	12,288	49,152	96	96	2,048
GPT-4	undisclosed	—	—	—	—	8,192–32,768

Same shape, four generations, **~1,500×** the parameters — and GPT-4's own hyperparameters are simply not public.

LLaMA: the same shape, modern plumbing

LLaMA (Large Language Model Meta AI) keeps GPT's decoder-only, causal shape — but swaps in modern components:

- **Position: RoPE** — rotary position embeddings.
- **Normalization: RMSNorm** — a cheaper layer-norm variant.
- **Activation: SwiGLU** in the FFN, a gated variant instead of ReLU.
- **Training:** BPE tokenizer, **FlashAttention**, `bfloat16` precision.
- **Attention**, LLaMA-2 onward: **grouped-query attention (GQA)**, which shrinks the inference-time memory footprint.

All named here, not derived today — Lesson 4 opens up several of them.

LLaMA's size ladder

Version	Parameters	Context	Training tokens	Notes
LLaMA-1	7–65B	2,048	1.4T	initial release
LLaMA-2	7–70B	4,096	2T	+ GQA
LLaMA-3	8–405B	8,192–128,000	15T	same architecture as v2

Training tokens grow **faster** than parameter count: 1.4T → 15T against 65B → 405B.

Remark: the Chinchilla scaling law

Read the LLaMA row across generations and one trend jumps out: **training tokens grow far faster than parameters** (1.4T → 15T vs 65B → 405B).

> That is the **Chinchilla** lesson: for a fixed compute budget, model size and training tokens should scale **together**. Empirically, most early large models were **undertrained** — too big in capacity relative to the tokens they saw.

Details — the compute-optimal frontier, the FLOPs-vs-parameters trade-off — are **beyond this module's scope**; see Hoffmann *et al.* (2022) in the references.

GPT: strengths & limitations

- ✅ **One objective, every task** — anything phrased as "continue this text" is trainable with the same next-token loss.
- ✅ **Dense signal**: every position is a training example — against MLM's ~15%.
- ✅ **Scales without redesign** — GPT-1 to GPT-4 is the same block, made wider, deeper, longer-context.
- ⚠️ **Left context only** — a token can never use what comes after it, which is exactly what reading tasks want.

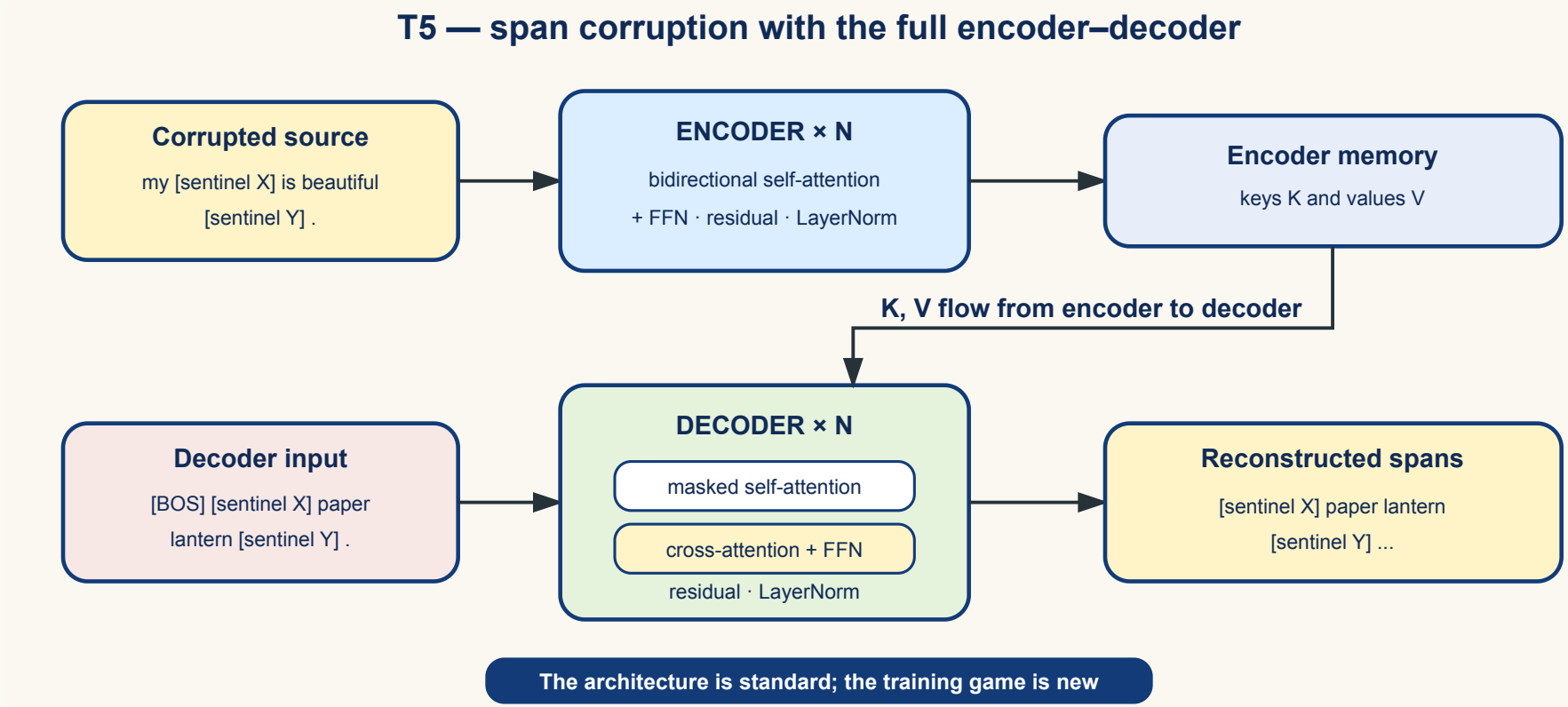
Part IV — T5: encoder-decoder

(c) T5: keep both, cast everything as text → text

Text-**T**o-**T**ext **T**ransfer **T**ransformer: the **full** original Transformer — encoder, decoder, cross-attention — with one unifying idea.

> **Every task is text → text.** Translation, summarization, even classification (`"cola sentence: … → acceptable"`) — one model, one output format: a string.

T5, drawn



T5's pretraining game: span corruption

A variant of MLM: mask out **contiguous spans** of tokens (not single tokens), each replaced by **one sentinel token**. The decoder's job is to reconstruct exactly those spans.

Input to the encoder

```
my <X> is beautiful <Y> .
```

Target for the decoder

```
<X> paper lantern <Y> was
```

> The encoder reads bidirectionally (like BERT); the decoder writes causally (like GPT) — span corruption suits this shape exactly, because reconstructing a *span* naturally needs generation, not a single fill-in.

T5's size ladder

Version	Parameters	d_{model}	d_{ff}	h	N
Small	60M	512	2,048	8	6
Base	220M	768	3,072	12	12
Large	770M	1,024	4,096	16	24
XL	3B	1,024	16,384	32	24
XXL	11B	1,024	65,536	128	24

Growth mostly widens d_{ff} and h; d_{model} is fixed at 1,024 from Large upward. `t5-small` (60M) is what the homework fine-tunes.

ByT5 — no tokenizer at all

Byte-level T5: instead of a word/subword vocabulary, work directly on **raw bytes**. Vocabulary size $2^8 = 256$ — every possible byte value, so it can represent **any** Unicode text.

`A cute teddy bear.` → `65 32 99 117 116 101 32 116 101 100 100 121 32 98 101 97 114 46`

Each number is one byte's value (a space is byte 32); a 4-byte emoji sequence (🐻 → `240 159 167 184`) works exactly the same way — **no `[UNK]` token is ever needed.**

ByT5: robust, but longer

Robust to typos

`Bear` → bytes for `bear` (correct). A misspelling `Beaar` still decodes to something close — its bytes mostly overlap with the correct spelling.

A **word-level** tokenizer instead maps the same typo straight to `[UNK]` — total information loss.

The cost: length

$$n_{\text{byte-level}} \gg n_{\text{word-level}}$$

"teddy bear" is **2** word-level tokens but **~18** bytes — and attention is **quadratic** in sequence length (Lesson 4).

⚠️ Robustness and vocabulary simplicity are bought with **sequence length** — nothing here is free.

BART — encoder-decoder denoising, four ways

Bidirectional **A**uto-**R**egressive **T**ransformer: another encoder–decoder, trained by adding **noise** to the input and reconstructing the clean version. BART mixes **four** noise strategies:

- **Token masking** — random tokens replaced with `[MASK]` (like BERT).
- **Sentence permutation** — sentences within the document shuffled.
- **Text infilling** — reconstruct a missing span from the input (like T5's span corruption).
- **Document rotation** — a random token becomes the document's "start"; the model must find where it really starts.

> One encoder-decoder shape, **four different ways to break the input** — BART's pretraining is a superset of BERT's and T5's noising ideas.

T5: strengths & limitations

✅ Strengths

- **One interface for everything** — one head, one output format, one loss.
- **Span corruption fits the shape**: encoder reads corrupted text, decoder writes the missing spans.
- **Cross-attention keeps the source available** at every decoding step.

⚠️ Limitations

- **You pay for both stacks** — encoder *and* decoder *and* cross-attention.
- **Task framing is a design decision** — results depend on how the prefix is phrased.
- **ByT5's robustness costs length** — and length is quadratic (Lesson 4).

⚠️ **Misconception:** *"T5 needs a different head per task."* No — **one** text decoder for everything; the task is named **inside the input string**.

Part V — The family tree, tabulated

The family tree, tabulated

Family	Keeps	Attention	Pretraining objective	Best at	Examples
BERT	encoder	bidirectional	MLM (+NSP)	understanding, classification	DistilBERT, RoBERTa
GPT	decoder	causal (masked)	next-token LM	generation, prompting	GPT-3, LLaMA
T5	encoder + decoder	bi (encoder), causal + cross (decoder)	span corruption	seq-to-seq, unified interface	mT5, BART

Shape decides the attention pattern; the objective decides the job. **Mixture-of-experts is not a fourth row** — it is orthogonal, and can be dropped into *any* of the three (Switch is a T5 with expert FFNs).

Part VI — Mixture-of-experts

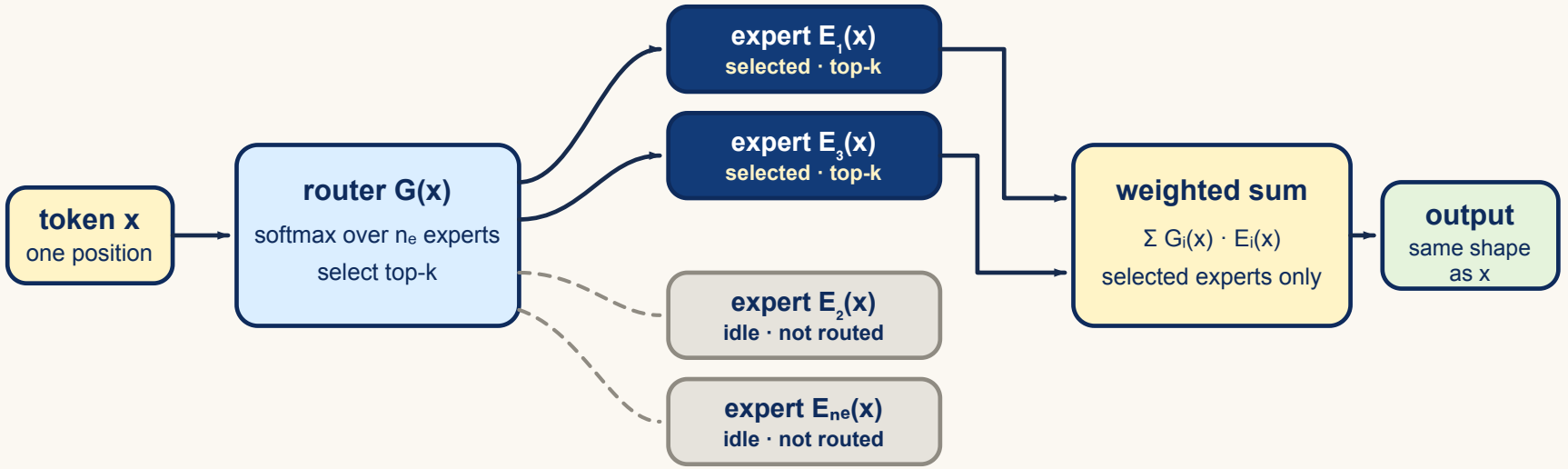
MoE: replace the FFN, not the attention

Lesson 2's own board exercise found the split inside one encoder block: attention ≈ 1.05M vs FFN ≈ 2.10M — the **FFN holds about two thirds** of the block. Mixture-of-experts attacks exactly that:

> **Replace the block's single FFN with n_e expert FFNs**, guarded by a **gate/router** $G(\cdot)$.
> Orthogonal to BERT/GPT/T5's axis — it can decorate any of the three.

$E_1, \ldots, E_{n_e}$ are the experts; n_e is the **expert count** (⚠ *not* the sequence length n from Lesson 1).

MoE, drawn



Only the selected experts (solid) do any work; the greyed-out ones cost **nothing** for this token.

MoE routing, step by step

Step 1 — score the experts. The router is a softmax over experts, $G(x) \in \mathbb{R}^{n_e}$: entry $G(x)_i$ says how much expert i should handle token x, and $\sum_i G(x)_i = 1$.

Step 2 — combine. Run *every* expert and blend (dense), or only the **top-k** (sparse):

$$\underbrace{\hat{y} = \sum_{i=1}^{n_e} G(x)_i \, E_i(x)}_{\text{densely gated}} \qquad \underbrace{\hat{y} = \sum_{i \in \mathcal{I}_k} G(x)_i \, E_i(x)}_{\text{sparsely gated, top-}k} \tag{1}$$

$\mathcal{I}_k$ is the index set of the k highest-scoring experts. The **Switch Transformer** takes the extreme case $k = 1$: exactly **one** expert per token.

Made concrete: 64 experts, one active

One encoder layer at Lesson 2's base configuration ($d_{\text{model}}=512$, $d_{\text{ff}}=2048$), with $n_e = 64$ experts and top-1 routing:

Per encoder layer	Dense block	MoE block ($n_e = 64$, $k = 1$)
Attention parameters	1.05M	1.05M
FFN parameters	2.10M (one FFN)	$64 \times 2.10\text{M} =$ **134.2M**
Total parameters	**3.15M**	**135.3M — 43×** more
Compute per token	3.15M-worth	**3.15M-worth — unchanged**

Only $k/n_e = 1/64$ of the experts fire: the FLOPs of a $\frac{k}{n_e}p$-parameter model. Switch crossed a **trillion** parameters this way.

MoE grows capacity, not speed: it makes the model bigger at similar per-token compute.

MoE's catch: the router must stay balanced

Left alone, the router **collapses** — a few experts win every token, the rest never train. Training adds an auxiliary loss over a batch $\mathcal{B}$ of T tokens:

$$\mathcal{L}_{\text{aux}} = \alpha\, n_e \sum_{i=1}^{n_e} f_i\, P_i, \qquad f_i = \frac{1}{T} \sum_{x \in \mathcal{B}} \mathbb{1}\left\{\arg\max p(x) = i\right\}, \qquad P_i = \frac{1}{T} \sum_{x \in \mathcal{B}} p_i(x) \tag{2}$$

f_i — the **hard** term: fraction of tokens *actually routed* to expert i. P_i — the **soft**, differentiable term: the router's *mean probability* for expert i.

What α is for. The network minimizes $\mathcal{L}_{\text{task}} + \mathcal{L}_{\text{aux}}$ — two goals at once, and $\alpha \sim 10^{-2}$ sets the exchange rate. **Too large** and the model buys a tidy routing table at the cost of accuracy; **too small** and the router collapses anyway. It is a *tie-breaker* among near-equal routings, not a goal in itself.

Made concrete: what imbalance costs

$n_e = 4$ experts, batch $T = 100$ tokens, evaluating (2):

Routing	f	P	$\sum_i f_i P_i$	$\mathcal{L}_{\text{aux}}$
Balanced (25 each)	$(.25, .25, .25, .25)$	$(.25, .25, .25, .25)$	$4(.0625) = 0.25$	$\alpha \cdot 4 \cdot 0.25 = \boldsymbol{\alpha}$
Skewed (70/10/10/10)	$(.70, .10, .10, .10)$	$(.55, .15, .15, .15)$	$.385 + .045 = 0.43$	$\mathbf{1.72}\,\boldsymbol{\alpha}$
Collapsed (100/0/0/0)	$(1, 0, 0, 0)$	$(.85, .05, .05, .05)$	0.85	$\mathbf{3.4}\,\boldsymbol{\alpha}$

The factor n_e **normalizes**: under perfectly uniform routing $n_e \sum_i f_i P_i = n_e \cdot n_e \cdot \frac{1}{n_e^2} = 1$, so $\mathcal{L}_{\text{aux}} = \alpha$ exactly, whatever n_e is — collapse costs at most $\alpha\, n_e$.

Even balanced, leave head-room: expert capacity

Routing is never perfectly uniform, so each expert gets a hard cap on tokens per batch:

$$\mathcal{E} = \frac{T}{n_e}\,\mathcal{C}, \qquad \mathcal{C} \geq 1 \tag{3}$$

T/n_e is the **fair share**; $\mathcal{C}$ is the **capacity factor** — slack for uneven routing. With $T{=}100$, $n_e{=}4$, $\mathcal{C}{=}1.25$: cap is $\mathcal{E} = 31.25 \to 31$ tokens per expert.

Why a hard cap? Experts sit on **different devices**; each buffer is sized *before* the batch is routed and cannot grow on the fly.

> ⚠ **Token 32 is dropped** — no second-choice expert, no queue. It **skips the FFN sublayer**, and the residual carries it through unchanged to the next layer.

The trade-off in $\mathcal{C}$: too small drops tokens; too large reserves buffers that sit empty.

MoE in practice: an application, and the family

A natural application — a translation model where each expert specializes in a subset of similar languages: Latin languages (French, Spanish, Italian), Germanic languages (German, Dutch), and so on.

Modern MoEs

- **Switch Transformer** — the $k=1$ extreme, past a trillion parameters.
- **GLaM**, **Mixtral** — production-scale sparse MoE LLMs.

MoE: strengths & limitations

✅ Strengths

- **Capacity decoupled from compute** — 43× the parameters at the same per-token cost.
- **Expert specialization** — different experts settle on different kinds of token.
- **Easy to grow**: add experts without touching the rest of the block.

⚠️ Limitations

- **Routing and hardware complexity** — experts spread across devices, tokens shipped to them.
- **Memory for *all* experts**, even though only k run per token.
- **Two more knobs**: the balance weight α in (2) and the capacity factor C in (3).

Part VII — Pick the family

Your turn — pick the family

> For each task, pick **BERT**, **GPT** or **T5** — and give **one sentence why**: attention pattern + pretraining objective + readout.

- sentiment classification
- English → Spanish translation
- code autocomplete
- named-entity recognition (NER)
- summarization
- fill-in-the-blank

One of the six is **genuinely ambiguous**. Find it, and argue both sides.

Answers

- **sentiment** → **BERT** — bidirectional, `[CLS]` head.
- **EN → ES** → **T5** — seq-to-seq by shape; GPT can too, by prompting.
- **code autocomplete** → **GPT** — left-to-right generation, literally its objective.
- **NER** → **BERT** — per-token labeling head.
- **fill-in-the-blank** → **BERT** — this *is* MLM.
- **summarization** → **T5** by shape *or* **GPT** by prompting — **both defensible**.

> **The ambiguous one is the point.** As models scale, decoder-only GPT-style models absorb tasks that once "belonged" to a specific shape — the boundaries are **conventions, not laws**.

Part VIII — Homework

The homework launches here

> No in-class lab today — this session's hands-on slot **is** the homework briefing. Assigned now, due before the end of the module.

Format: the same sandbox convention as every other lab — machinery provided and already running, exercises ask you to *investigate* it. **Part 0** loads and wraps **three real, pretrained models**, one per family:

Model	Family	Parameters
`distilbert-base-uncased`	encoder-only	66M
`gpt2`	decoder-only	124M
`t5-small`	encoder-decoder	60M

Five experiments, **H1–H5**, **60 points — all on the written conclusions**, none on code.

H1 & H2 — what pretraining objective buys you

H1 — same task, three native answers *(11 pts)*. Run **≥ 6** cloze-style sentences through each model's own native format (`bert_fill` , `gpt2_next` on a left-truncated prompt, `t5_fill`). Find a sentence where the correct fill needs **right-context** — only the bidirectional models can use it.

H2 — does the future leak into the past? *(12 pts)*. Build **≥ 4** sentence pairs identical up to a target token, differing only **after** it. Measure how much the target's hidden state moves, for BERT **and** GPT-2, across **≥ 3** layers. GPT-2's causal mask makes its answer **provably zero** — an exact, deductive fact, not a guess.

H3 & H4 — fixed budget, and a router under load

H3 — one task, one fixed budget, three architectures *(14 pts, the highest-value answer).* Fine-tune **all three** on the same 300/100 SST-2 slice, same budget. BERT on `[CLS]`, GPT-2 on its last token (needs the pad-token fix), T5 as **text generation** (predict the literal string `"positive"` / `"negative"`). Compare accuracy — and connect the result back to H1/H2's mechanistic findings.

H4 — what does an unbalanced router actually cost? *(13 pts).* A toy MoE layer (eqs. 1–3, just taught): route tokens through an **untrained** router, then a **deliberately skewed** one — measure $f_i, P_i, \mathcal{L}_{\text{aux}}$ **and** the token **drop rate** at capacity. Then add the auxiliary loss and watch both fall.

H5 — synthesis, and how it's graded

> **H5 — synthesis** *(10 pts, 200–350 words)*. Tie together: what each objective buys/costs (grounded in **your own** H1/H2 evidence); whether H3's numbers matched the mechanistic prediction; what H4 showed about the cost of imbalance; a "which family for which job" recommendation citing ≥ 2 experiments; and **one claim your evidence does not support.**

H1	H2	H3	H4	H5	Total
11	12	14	13	10	**60**

The experiments must **run** — every `✓` check prints the numbers a conclusion has to cite — but **no points for code.**

Recap

- **One design, three shapes.** The same attention+FFN+residual/LN recombine by *which halves of the encoder-decoder you keep* and *which pretraining game you play*.
- **BERT** (encoder, bidirectional) plays **MLM(+NSP)** — reads, cannot generate; DistilBERT/RoBERTa/ALBERT/ELECTRA each bet on a different axis of that recipe.
- **GPT** (decoder, causal) plays **next-token LM** — dense signal, scales without redesign; LLaMA keeps the shape, swaps the plumbing (RoPE, RMSNorm, SwiGLU — Lesson 4 opens these up).
- **T5** (encoder+decoder) plays **span corruption**, casting every task as text → text; ByT5 trades vocabulary for length, BART mixes four kinds of noise.
- **Mixture-of-experts** is *orthogonal*: swap one FFN for n_e experts, gated by $G(\cdot)$, decoupling capacity from compute — but the router needs a **load-balancing loss** and a **capacity cap** or it collapses.

Concept check

A colleague claims: *"Since T5 has the biggest architecture, it should always beat BERT and GPT on any task."*

Using today's compare-and-contrast table, what is wrong with that claim?

What's next

Lesson 4 — Layer tricks, speedups, interpretability. We open the residual+LN wrapper you've been reading since Lesson 2 and finally derive it, add label smoothing, attack attention's $O(n^2)$ wall (sparse, low-rank, FlashAttention), name GQA/MQA properly, survey interpretability's promises and limits (attention ≠ explanation), and cover two short new topics — the KV cache and a first look at LoRA.

> **Going deeper before then?** Read the family you're least sure about in the homework's H1 output — does its native behavior match what today's table predicted?

References & further reading (1/3)

[1] A. Amidi and S. Amidi, *Super Study Guide: Transformers and Large Language Models*, Part 3, Sec. 3.4, pp. 98–113, 2024.

[2] J. Devlin *et al.*, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[3] A. Radford *et al.*, "Improving Language Understanding by Generative Pre-Training," OpenAI, Tech. Rep., 2018.

[4] A. Radford *et al.*, "Language Models Are Unsupervised Multitask Learners," OpenAI, Tech. Rep., 2019.

[5] C. Raffel *et al.*, "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.

References & further reading (2/3)

[6] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity," *J. Mach. Learn. Res.*, vol. 23, pp. 1–39, 2022.

[7] N. Shazeer *et al.*, "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer," in *Proc. ICLR*, 2017.

[8] V. Sanh *et al.*, "DistilBERT, a Distilled Version of BERT," arXiv:1910.01108, 2019.

[9] Y. Liu *et al.*, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv:1907.11692, 2019.

[10] Z. Lan *et al.*, "ALBERT: A Lite BERT for Self-Supervised Learning of Language Representations," in *Proc. ICLR*, 2020.

References & further reading (3/3)

[11] K. Clark *et al.*, "ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators," in *Proc. ICLR*, 2020.

[12] H. Touvron *et al.*, "LLaMA: Open and Efficient Foundation Language Models," arXiv:2302.13971, 2023.

[13] M. Lewis *et al.*, "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation," in *Proc. ACL*, 2020, pp. 7871–7880.

[14] J. Hoffmann *et al.*, "Training Compute-Optimal Large Language Models," in *Advances in Neural Information Processing Systems 35*, 2022.

[15] J. Alammar, "The Illustrated Transformer," 2018. [Online]. Available: https://jalammar.github.io/illustrated-transformer/